Manual técnico: Auditor y Auto-Remediador de Seguridad Linux

1. Objetivo y alcance

Este proyecto realiza una inspección rápida de configuraciones críticas habituales en un servidor Linux y emite evidencia legible por personas (Markdown) y máquinas (JSON). Su referencia es un subconjunto educativo de prácticas CIS, llamado aquí “CIS ligero”.

El alcance es deliberadamente reducido: permisos de archivos de cuentas, configuración efectiva de SSH, servicios heredados y firewall. No afirma conformidad con CIS Benchmark ni reemplaza un análisis de vulnerabilidades, una gestión de parches o una política de seguridad.

2. Arquitectura

El ejecutable securityauditor.py tiene cuatro etapas:

1. Descubrimiento: lee metadatos locales y ejecuta comandos sin shell=True.
2. Evaluación: convierte cada resultado en un hallazgo PASS, FAIL, WARN o INFO.
3. Reporte: serializa los hallazgos y el resumen a JSON o Markdown.
4. Remediación opcional: solo se habilita al indicar un control y la confirmación --yes.

No transmite datos ni instala software. El modo de auditoría no requiere privilegios de root, aunque puede tener menor visibilidad sobre algunos recursos.

3. Controles evaluados

3.1 Archivos de cuentas

ID Revisión Criterio de aprobación Riesgo mitigado
|---|---|---|---|
CIS-LITE-001 /etc/passwd Archivo regular, UID 0 y permisos como máximo 0644 Alteración de cuentas y hashes de contraseña
CIS-LITE-002 /etc/shadow Archivo regular, UID 0 y permisos como máximo 0640; grupo autorizado para lectura Exposición de hashes de contraseña

Los modos son máximos, no valores obligatorios: 0400 y 0600 se conservan. Se rechazan ejecución, bits especiales y escritura de grupo/otros. Para shadow, la lectura de grupo se admite con GID 0 o grupo local shadow resuelto mediante NSS; con otro grupo se exige como máximo 0600. No se cambia el GID. Los enlaces, archivos no regulares y metadatos/grupos inaccesibles producen WARN sin remediación automática.

3.2 SSH

ID Parámetro efectivo Aprobación Tratamiento
|---|---|---|---|
CIS-LITE-003 PermitRootLogin no Puede remediarse de forma controlada
CIS-LITE-004 PasswordAuthentication no Solo recomendación manual

La consulta se realiza con sshd -T -f /etc/ssh/sshdconfig, que evalúa los valores que aplica el daemon. Si hay una configuración con sintaxis inválida, el resultado será WARN; corrija primero el archivo y ejecute sshd -t.

3.3 Servicios heredados

Se detecta el estado activo de telnet.socket, rsh.socket, rexec.socket, tftp.socket y xinetd.service mediante systemd. La combinación de código 0 con salida active produce FAIL; código 3 con inactive o failed produce PASS respecto del criterio de actividad. Estados desconocidos, transitorios, unidades no identificadas, ausencia de systemctl o errores de consulta producen WARN. Esto no demuestra que el software esté desinstalado ni que no haya procesos fuera de estas unidades.

La herramienta no deshabilita servicios: su eliminación puede afectar aplicaciones antiguas que requieren una autorización explícita.

3.4 Firewall

- Si UFW existe, la comprobación aprueba solo si ufw status informa Status: active. Un error, estado vacío, desconocido o contradictorio produce WARN; una salida válida de firewall inactivo produce FAIL respecto de UFW.
- Si UFW no existe pero iptables sí, aprueba sólo si la consulta termina correctamente y contiene una única política por defecto INPUT DROP. REJECT no es una política por defecto válida; políticas desconocidas, duplicadas, ausentes o ACCEPT producen WARN, pues no se evalúa el conjunto completo de reglas.
- Si no existe ninguno, informa WARN.

Una política restrictiva por sí sola no demuestra que las reglas sean correctas. En particular, esta versión no analiza nftables, firewalld ni reglas IPv6.

4. Ejecución y reportes

`bash
python3 securityauditor.py --format json --output reports/host.json
python3 securityauditor.py --format markdown --output reports/host.md
`

Campos principales del JSON:

- generatedat, host, platform: contexto de la ejecución.
- summary: número de resultados por estado.
- findings: ID, título, severidad, evidencia, recomendación y si es remediable.

Los hallazgos tienen estas interpretaciones:

Estado Significado Acción esperada
|---|---|---|

PASS El criterio revisado se cumple Conservar como evidencia
FAIL Se detectó una configuración insegura Revisar, aprobar y corregir
WARN No se cumple una recomendación o no se pudo concluir Analizar el contexto
INFO Información contextual o inaplicabilidad explícitamente establecida Revisar el
alcance

El programa usa 0 si no detecta fallos, 1 si hay al menos un FAIL y 2 si falla una
remediación. 0 no garantiza que todos los controles hayan podido evaluarse. Los errores de
ejecución, herramientas ausentes y datos incompletos producen WARN, no INFO ni un PASS por
defecto.

Los comandos se ejecutan con LCALL=C, LANG=C y LANGUAGE=C en su entorno hijo, manteniendo
sin cambios el entorno del proceso padre. El tiempo máximo de consulta es 12 segundos; un
timeout se registra internamente con código 124 y un error al iniciar el comando con 127.
SSH informa ambos controles como WARN cuando la consulta efectiva falla, y no ofrece
remediar valores ausentes o desconocidos.

5. Remediación y protección contra bloqueos

No existe una opción “arreglar todo”. Se debe elegir un control y confirmar el cambio:

`bash
sudo python3 securityauditor.py --remediate permissions --yes
sudo python3 securityauditor.py --remediate ssh-root-login --yes
`

permissions comprueba ambos archivos antes de modificar, rechaza enlaces y archivos no
regulares, establece UID root solo cuando corresponde y elimina únicamente permisos
excedentes. Conserva los permisos más restrictivos y el grupo; no lee contenidos. Usa
descriptores y verifica los metadatos después del cambio. No ofrece rollback
transaccional: si una operación falla se informa qué cambios anteriores se realizaron.
ssh-root-login rechaza configuraciones con Match o Include para exigir revisión manual de
las excepciones. En configuraciones simples crea una copia privada con modo 0600 y marca
temporal con microsegundos, aplica PermitRootLogin no al principio y ejecuta sshd -t y
sshd -T. Si falla la validación o el valor efectivo no es no, restaura el contenido
original. No recarga el servicio: el operador debe recargarlo y probar una nueva conexión
antes de cerrar su sesión existente.

La auditoría con sshd -T consulta valores globales; no evalúa todas las combinaciones
condicionales de Match mediante -C. Un PASS global no garantiza el mismo resultado para
cada usuario o dirección de origen.

No se cambia automáticamente PasswordAuthentication: una configuración incorrecta de llaves
puede bloquear al operador. Antes de desactivarla, mantenga una segunda sesión abierta,
confirme que una cuenta no-root accede por llave y valide la configuración con sudo sshd
-t.

Tampoco se aplican reglas de firewall ni se deshabilitan servicios porque esos cambios

requieren conocer puertos, dependencias y ruta de acceso de cada host.

6. Operación en Vagrant / home lab

Ejemplo de provisión que solo genera evidencia:

```bash
sudo install -m 0755 /vagrant/securityauditor.py /usr/local/sbin/securityauditor.py
sudo mkdir -p /var/log/security-auditor
sudo python3 /usr/local/sbin/securityauditor.py \
--format json --output /var/log/security-auditor/initial.json || true
```

El || true evita que un hallazgo de endurecimiento corte el provisionamiento; el reporte sigue disponible para revisión. Para producción, conecte el JSON a su sistema de inventario, SIEM o pipeline, y establezca una política explícita para tratar resultados FAIL.

7. Buenas prácticas posteriores

1. Haga copia o snapshot de la VM antes de cambiar SSH/firewall.
2. Pruebe en una VM equivalente antes que en producción.
3. Documente excepciones y su fecha de revisión.
4. Compruebe el acceso tras cada modificación.
5. Complete la evaluación con actualizaciones, MFA/bastion, logs, backups, SELinux/AppArmor, nftables/firewalld y escáneres de vulnerabilidades.

8. Generación del PDF

MANUAL.pdf es la versión imprimible de este documento. Para regenerarlo con Python:

```bash
python3 tools/buildpdf.py docs/MANUAL.md docs/MANUAL.pdf
```

El script utiliza solo la biblioteca estándar de Python y genera un PDF básico, por lo que no requiere Pandoc ni conexión a Internet.